

Neural Networks

Lecture 3 — Backpropagation

```
loss.backward()
```

One line. What is it actually doing?

The question

In the training loop you wrote:

```
loss.backward()    # ← this one
with torch.no_grad():
    w -= lr * w.grad
```

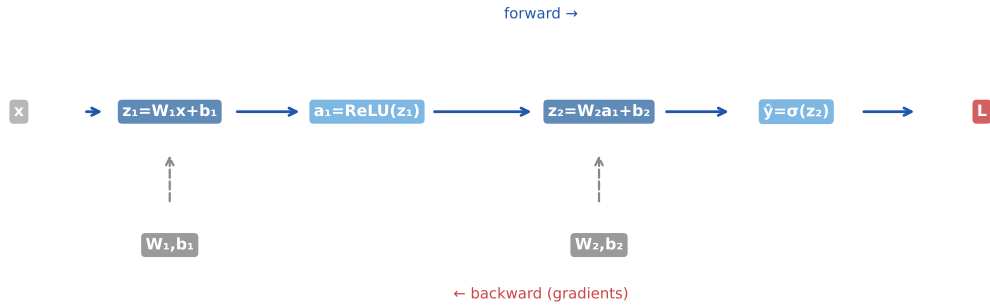
`w.grad` appeared from nowhere and contained the right number.

For a single linear layer the gradient is easy to derive by hand. For a 10-layer network with millions of parameters it is not.

Backpropagation is the algorithm that computes all gradients efficiently — one pass backward through the network, reusing intermediate results.

The computation graph

Every PyTorch operation on a `requires_grad` tensor is recorded in a computation graph.



Forward pass: data flows left $\rightarrow$ right. **Backward pass:** gradients flow right $\rightarrow$ left via chain rule. PyTorch builds this graph dynamically — every forward pass recreates it.

Chain rule — one layer

For a single linear layer followed by a loss:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad L = \frac{1}{N} \|\hat{\mathbf{y}} - \mathbf{y}\|^2$$

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{w}} = \frac{2}{N} X^\top (\hat{\mathbf{y}} - \mathbf{y})$$

The chain rule factors the gradient into two terms: how sensitive is the loss to the output, and how sensitive is the output to the weights?

Chain rule — two layers

$$z_1 = W_1 \mathbf{x} + b_1, \quad a_1 = \text{ReLU}(z_1), \quad z_2 = W_2 a_1 + b_2, \quad \hat{y} = \sigma(z_2)$$

Backward:

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \cdot \sigma'(z_2)$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2}^\top a_1 \quad \frac{\partial L}{\partial a_1} = \frac{\partial L}{\partial z_2} W_2$$

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \cdot \mathbf{1}[z_1 > 0] \quad \frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1}^\top \mathbf{x}$$

Each layer's gradient depends only on the local derivative and the gradient arriving from the layer above. This is the key insight — gradients chain.

Autograd: PyTorch does this for you

```
import torch

x = torch.tensor([[0.75, 2.1]])      # one galaxy: g-r, r-i
W1 = torch.randn(2, 4, requires_grad=True)
b1 = torch.zeros(4, requires_grad=True)
W2 = torch.randn(4, 1, requires_grad=True)
b2 = torch.zeros(1, requires_grad=True)

# Forward
a1 = torch.relu(x @ W1 + b1)
y_hat = torch.sigmoid(a1 @ W2 + b2)
loss = ((y_hat - torch.tensor([[1.0]])) ** 2).mean()

# Backward - one call computes all four gradients
loss.backward()

print(W1.grad.shape)  # (2, 4) - correct
print(W2.grad.shape)  # (4, 1) - correct
```

The same mechanism works for 100 layers, skip connections, and arbitrary architectures.

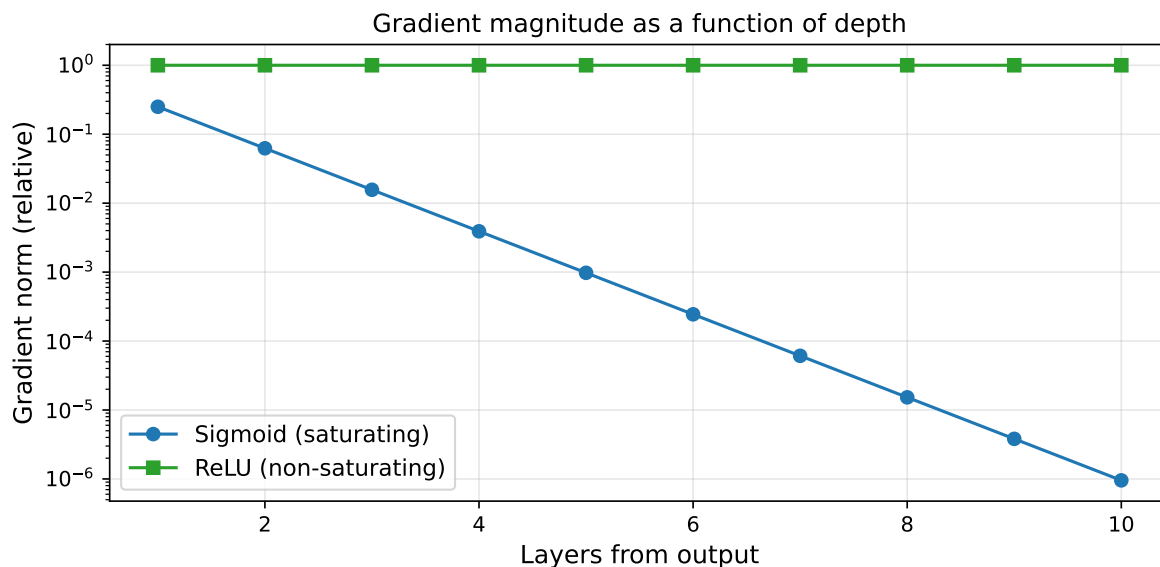
The vanishing gradient problem

Each backward step multiplies by a local derivative $\sigma'(z)$.

For sigmoid: $\sigma'(z) \in (0, 0.25)$ — always less than 1.

In a deep network, the gradient at layer k from the top is:

$$\frac{\partial L}{\partial W_k} \propto \prod_{i=k}^L \sigma'(z_i)$$



After 10 sigmoid layers, the gradient at the first layer is $\approx 10^{-6}$ times the output gradient. The first-layer weights barely update. **This is why deep networks with sigmoid activations were nearly untrainable before ReLU.**

Why ReLU works

$$\text{ReLU derivative: } \sigma'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

For an active neuron ($z > 0$), the gradient passes through **unchanged** — no shrinking. For an inactive neuron ($z \leq 0$), the gradient is zero — the dead neuron problem.

In practice, only a fraction of neurons are dead at any time. The active neurons carry gradients cleanly back to the earliest layers.

This is why ReLU — not sigmoid or tanh — is the default for hidden layers in modern networks.

Open the notebook.

You will compute gradients by hand for a 2-layer network, then verify them against PyTorch's autograd — and watch gradients vanish as depth increases.

References

Papers

- Rumelhart, Hinton & Williams (1986) — *Learning representations by back-propagating errors*. Nature 323, 533–536
- Hochreiter (1991) — *Untersuchungen zu dynamischen neuronalen Netzen*. Original vanishing gradient analysis. Summary: arxiv.org/abs/1503.04069

Videos & blogs

- 3Blue1Brown — *Backpropagation, step by step* youtube.com/watch?v=Ilg3gGewQ5U
- Karpathy — *Yes you should understand backprop* karpathy.medium.com/yes-you-should-understand-backprop-e2f06eab496b